

UTS
PENGOLAHAN CITRA DIGITAL



INTELLIGENT **COMPUTING**

NAMA : MUHAMMAD RAKA ILHAM

NIM : 202431145

KELAS : A

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC : 15

ASISTEN : 1. Sharira Maharani

2. Muhammad Hanif Nuruddin Jabbar

3. Fhazel Kesra Arivi

4. Muhammad Farhan Fahrezy

INSTITUT TEKNOLOGI PLN
TEKNIK INFORMATIKA

2025/2026

DAFTAR ISI

Contents

DAFTAR ISI	2
BAB I	3
PENDAHULUAN	3
1.1 Rumusan Masalah	3
1.2 Tujuan Masalah	3
1.3 Manfaat Masalah	3
BAB II	4
LANDASAN TEORI	4
2.1 Citra Digital	4
2.2 Jenis-Jenis Citra Digital	4
2.2.1 Citra Berwarna (RGB)	4
2.2.2 Citra Grayscale	4
2.3 Ruang Warna HSV	5
2.4 Operasi Piksel: Kecerahan dan Kontras	5
2.4.1 Penyesuaian Kecerahan (<i>Brightness Adjustment</i>)	5
2.4.2 Penyesuaian Kontras (<i>Contrast Adjustment</i>)	5
2.5 Histogram Citra	6
2.6.1 OpenCV	6
2.6.2 NumPy	6
2.6.3 Matplotlib	6
BAB III	7
HASIL	7
3.1 Soal 1 Manipulasi Warna dan Segmentasi Warna	7
3.1.1 Pembacaan Citra dan Konversi BGR ke RGB	7
3.1.2 Manipulasi Kontras Citra Warna ($\alpha = 1.2$)	7
3.1.3 Segmentasi Warna Menggunakan Masking HSV	8
3.1.4 Kombinasi Masking (Citra Biner)	10
3.2 Soal 2 Operasi Piksel pada Citra Grayscale	12
3.2.1 Pembacaan Citra dan Konversi ke Grayscale	12
3.2.2 Citra Grayscale dan Visualisasi Histogram	13
3.2.3 Penyesuaian Kecerahan ($\beta = 20$) Manual Loop	13
BAB IV	18
PENUTUP	18
4.1 Kesimpulan	18

BAB I PENDAHULUAN

1.1 Rumusan Masalah

Pengolahan citra digital merupakan bidang ilmu yang berperan penting dalam pemrosesan dan analisis gambar secara komputasional. Dalam praktikum UTS Pengolahan Citra ini, terdapat beberapa permasalahan yang menjadi fokus kajian, yaitu:

- Bagaimana cara melakukan manipulasi warna pada citra digital menggunakan ruang warna RGB dan HSV dengan library OpenCV?
- Bagaimana cara melakukan segmentasi warna (biru, merah, dan hijau) pada citra menggunakan metode masking berbasis rentang nilai HSV?
- Bagaimana pengaruh kombinasi operasi masking terhadap hasil isolasi warna pada citra?
- Bagaimana cara melakukan operasi piksel pada citra grayscale, meliputi penyesuaian kecerahan (*brightness*) menggunakan nilai beta dan penyesuaian kontras (*contrast*) menggunakan nilai alpha?
- Bagaimana pengaruh parameter alpha dan beta terhadap distribusi histogram pada citra grayscale?

1.2 Tujuan Masalah

Berdasarkan rumusan masalah yang telah dijabarkan di atas, praktikum UTS ini bertujuan untuk:

- Mampu melakukan konversi ruang warna dari BGR ke RGB dan dari BGR ke HSV menggunakan OpenCV.
- Mampu mengimplementasikan segmentasi warna pada citra menggunakan teknik masking HSV untuk mengisolasi warna biru, merah, dan hijau secara terpisah maupun gabungan.
- Mampu menerapkan operasi piksel secara manual (iterasi per piksel) maupun secara vektorisasi menggunakan NumPy untuk mengubah kecerahan dan kontras pada citra grayscale.
- Mampu menganalisis perubahan distribusi intensitas piksel melalui visualisasi histogram sebelum dan sesudah manipulasi citra.
- Mampu menerapkan fungsi `np.clip()` untuk membatasi nilai piksel agar tetap dalam rentang $[0, 255]$

1.3 Manfaat Masalah

Praktikum UTS Pengolahan Citra ini memberikan manfaat sebagai berikut:

- **Bagi mahasiswa**, praktikum ini meningkatkan pemahaman teoritis sekaligus kemampuan praktis dalam mengimplementasikan teknik-teknik dasar pengolahan citra digital menggunakan Python, OpenCV, dan NumPy.
- **Secara akademis**, hasil praktikum ini menjadi landasan yang kuat untuk memahami konsep-konsep lanjutan dalam pengolahan citra, seperti deteksi objek, segmentasi citra, dan computer vision.
- **Secara praktis**, kemampuan memanipulasi kecerahan, kontras, dan segmentasi warna dapat diaplikasikan pada berbagai domain nyata seperti sistem pengawasan (*surveillance*), analisis citra medis, dan pengenalan pola visual.
- Mahasiswa memperoleh pemahaman tentang perbedaan pendekatan pemrosesan secara manual (iterasi piksel) versus vektorisasi, serta implikasinya terhadap efisiensi komputasi

BAB II

LANDASAN TEORI

2.1 Citra Digital

Citra atau gambar merupakan sinyal dwimatra (2 dimensi) yang bersifat kontinu sehingga dapat diamati oleh sistem visual manusia. Secara matematis, citra dinyatakan sebagai intensitas cahaya pada bidang dwimatra yang dinyatakan dalam fungsi $f(x,y)$, di mana (x,y) merupakan koordinat pada bidang dwimatra dan $f(x,y)$ menyatakan intensitas cahaya pada titik tersebut.

Menurut Gonzalez & Woods (2022), suatu citra dapat didefinisikan sebagai fungsi dua dimensi $f(x,y)$, di mana koordinat spasial horizontal dan vertikal direpresentasikan oleh dua sumbu x dan y , sedangkan nilai intensitas cahaya atau warna pada setiap titik dalam citra direpresentasikan oleh nilai f yang berhingga. Ketika nilai x , y , dan intensitas f semuanya berhingga dan diskrit, citra yang dihasilkan disebut sebagai citra digital.

Citra digital terdiri dari piksel-piksel kecil yang membentuk grid atau matriks. Setiap piksel mewakili sebuah titik dalam citra dan memiliki nilai numerik yang menggambarkan intensitas cahaya atau warna pada titik tersebut. Nilai numerik yang direpresentasikan ini umumnya merupakan nilai biner 8 bit, yang disimpan pada elemen citra yang disebut sebagai piksel. Piksel ini disimpan pada memori komputer sebagai array dua dimensi bertipe integer.

2.2 Jenis-Jenis Citra Digital

2.2.1 Citra Berwarna (RGB)

Citra berwarna merupakan citra yang mempunyai 3 buah kanal warna di dalamnya. Pada umumnya jenis citra ini terbentuk dari komponen merah (*red/R*), hijau (*green/G*), dan biru (*blue/B*) yang dimodelkan ke dalam ruang warna RGB. RGB adalah standar yang digunakan untuk menampilkan citra berwarna pada layar televisi maupun layar komputer.

Citra berwarna atau *true color image* sering juga disebut dengan *24 bit color image* karena setiap nilai pikselnya memerlukan penyimpanan sebesar 24 bit. Masing-masing piksel pada setiap kanal mempunyai kemungkinan nilai sebanyak 256, yaitu dengan nilai di antara 0–255. Karena citra berwarna memiliki 3 kanal, maka untuk satu piksel memerlukan $3 \times 8 \text{ bit} = 24 \text{ bit}$, sehingga total kombinasi warna yang mungkin adalah $256 \times 256 \times 256 = 16.777.216$ variasi warna.

Perlu dicatat bahwa OpenCV membaca citra berwarna dalam format BGR (*Blue, Green, Red*), bukan RGB. Oleh karena itu, diperlukan konversi menggunakan `cv2.cvtColor()` sebelum ditampilkan dengan Matplotlib yang menggunakan format RGB.

2.2.2 Citra Grayscale

Citra *grayscale* atau skala keabuan merupakan citra yang mempunyai 1 buah kanal sehingga yang ditampilkan hanyalah nilai intensitas atau yang dikenal juga dengan istilah derajat keabuan. Karena citra jenis ini hanya mempunyai 1 kanal saja, maka citra ini mempunyai tempat penyimpanan yang lebih hemat dan disebut juga sebagai *8 bit color image* karena setiap nilai piksel memerlukan penyimpanan sebesar 8 bit.

Secara teori, terdapat beberapa cara dalam mengonversi citra berwarna menjadi citra skala keabuan. Cara yang paling mudah adalah dengan merata-ratakan semua nilai piksel RGB menggunakan persamaan:

$$y = \frac{1}{3}(R + G + B)$$

Namun untuk mendapatkan hasil yang lebih baik, dapat digunakan persamaan pembobotan:

$$y = 0,299R + 0,587G + 0,144B$$

2.3 Ruang Warna HSV

Selain RGB, salah satu ruang warna penting yang digunakan dalam pengolahan citra adalah HSV (*Hue, Saturation, Value*). Terdapat berbagai ruang warna yang dapat digunakan untuk merepresentasikan citra berwarna, di antaranya adalah CMYK (*Cyan, Magenta, Yellow, Black*), HSV (*Hue, Saturation, Value*), YCbCr, dan Lab.

Ruang warna HSV merepresentasikan warna dengan tiga komponen yang lebih intuitif dibandingkan RGB: *Hue* (H) menyatakan jenis warna dalam bentuk sudut 0° - 360° , *Saturation* (S) menyatakan tingkat kemurnian atau kepekatan warna dengan rentang 0-255, dan *Value* (V) menyatakan tingkat kecerahan dengan rentang 0-255. Keunggulan ruang warna HSV dalam pengolahan citra adalah kemampuannya memisahkan informasi warna dari informasi kecerahan, sehingga proses segmentasi warna menjadi lebih robust terhadap perubahan pencahayaan.

2.4 Operasi Piksel: Kecenderahan dan Kontras

Operasi piksel merupakan operasi fundamental dalam pengolahan citra yang bekerja langsung pada nilai setiap piksel. Parameter-parameter citra yang dapat dimanipulasi untuk mendapatkan kualitas citra yang lebih baik di antaranya adalah perbaikan kontras, perbaikan tepi objek, penajaman citra, penapisan derau, dan pemberian warna semu.

2.4.1 Penyesuaian Kecenderahan (*Brightness Adjustment*)

Intensity adjustment digunakan untuk mengubah intensitas histogram. Cara kerjanya ialah nilai intensitas dari histogram suatu citra diubah menjadi nilai intensitas yang baru menggunakan transformasi persamaan linear, yang ditulis secara matematis sebagai:

$$f(x, y)' = f(x, y) + b$$

dengan $f(x, y)'$ adalah citra setelah penyesuaian, $f(x, y)$ merupakan citra sebelum penyesuaian, sedangkan b merupakan konstanta faktor penyesuaian. Jika b positif, maka kecerahan bertambah dan jika b negatif, maka kecerahan gambar berkurang.

Apabila nilai piksel pada suatu citra ditambahkan, maka efek yang terjadi adalah citra tersebut akan bertambah grayscale-nya sehingga tampak lebih terang. Nilai maksimal pada citra adalah 255, sehingga jika hasil penambahan tersebut melebihi 255 maka akan ditetapkan menjadi 255 saja. Pembatasan nilai ini dalam implementasi Python menggunakan NumPy dilakukan dengan fungsi `np.clip()` yang memastikan nilai piksel tetap berada dalam rentang $[0, 255]$.

2.4.2 Penyesuaian Kontras (*Contrast Adjustment*)

Kontras pada citra merepresentasikan perbedaan intensitas antara bagian terang dan gelap. Penyesuaian kontras dilakukan dengan mengalikan nilai piksel dengan faktor skala (α/α):

$$g(x, y) = \alpha \cdot f(x, y)$$

di mana $\alpha > 1$ akan meningkatkan kontras dan $0 < \alpha < 1$ akan menurunkan kontras. Nilai hasil perkalian tetap perlu dibatasi pada rentang $[0, 255]$ menggunakan clipping agar tidak terjadi overflow.

2.5 Histogram Citra

Dalam pengolahan citra digital, histogram merupakan representasi grafis dari distribusi intensitas piksel dalam sebuah citra. Histogram menggambarkan jumlah piksel dengan intensitas tertentu pada sumbu Y (frekuensi) terhadap intensitas piksel pada sumbu X.

Dari histogram, dapat ditunjukkan banyak hal tentang kecerahan (*brightness*) dan kontras dari sebuah citra. Sumbu Y (vertikal) pada grafik histogram menunjukkan nilai piksel dari citra sedangkan sumbu X (horizontal) menunjukkan nilai derajat keabuan (*greyscale*) citra.

Informasi yang terkandung pada histogram dapat dijadikan penentuan tingkat kecerahan dan kontras suatu citra. Apabila lebar histogram terlalu sempit, maka citra tersebut memiliki kontras yang terlalu terang (*overexposed*) atau terlalu gelap (*underexposed*). Sedangkan apabila lebar histogram merata maka menunjukkan citra tersebut memiliki kualitas yang baik karena derajat keabuan penuh dan merata pada setiap nilai intensitas piksel.

Dari histogram suatu citra dapat diketahui intensitas citra secara keseluruhan sehingga dapat disimpulkan apakah citra tersebut gelap atau terang. Citra dengan intensitas terang akan mempunyai sebaran histogram yang cenderung ke kanan, sedangkan citra dengan intensitas gelap akan mempunyai sebaran histogram yang cenderung ke kiri. Semakin tinggi nilai rata-rata histogram, maka semakin tinggi kecerahan suatu citra.

Sebuah citra berwarna dapat didekomposisi menjadi citra yang banyaknya sesuai dengan jumlah kanal citra, dan masing-masing citra hasil dekomposisi tersebut mempunyai histogramnya masing-masing.

2.6 Library Python untuk Pengolahan Citra

2.6.1 OpenCV

OpenCV (*Open-Source Computer Vision Library*) adalah pustaka yang digunakan untuk pemrosesan dan analisis citra, seperti deteksi objek, deteksi wajah, segmentasi citra, dan pengenalan pola. OpenCV juga mendukung berbagai operasi pengolahan citra lainnya.

2.6.2 NumPy

NumPy (*Numerical Python*) adalah pustaka yang digunakan untuk komputasi numerik, di mana citra pada dasarnya merupakan array nilai piksel yang mana setiap piksel diwakili oleh nilai 1 (Grayscale) atau 3 (Red-Green-Blue). NumPy sangat berguna dalam pengolahan citra digital, terutama untuk operasi matematika pada array piksel seperti manipulasi nilai piksel.

2.6.3 Matplotlib

Matplotlib adalah pustaka visualisasi data Python yang digunakan untuk menampilkan citra dan histogram hasil pengolahan. Matplotlib dapat diintegrasikan dengan pustaka lain seperti NumPy untuk menganalisis dan memvisualisasikan data dalam berbagai bentuk seperti plot, histogram, dan diagram batang.

BAB III

HASIL

Pada praktikum UTS Pengolahan Citra Digital ini terdapat dua soal utama yang dikerjakan, yaitu Soal 1 mengenai manipulasi warna dan segmentasi warna pada citra berwarna, serta Soal 2 mengenai operasi piksel pada citra grayscale. Citra yang digunakan adalah manipulasi_warna.jpeg berukuran 1600×1201 piksel dan RAKA.jpeg berukuran 1280×720 piksel.

3.1 Soal 1 Manipulasi Warna dan Segmentasi Warna

Citra yang digunakan: manipulasi_warna.jpeg — foto kertas bertulis nama "MUHAMMAD" (tinta hijau), "Raka" (tinta merah), dan "ILHAM" (tinta biru).

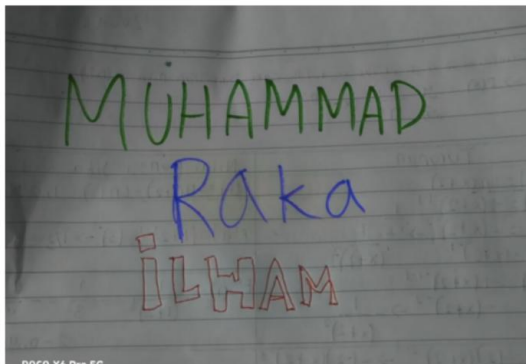
3.1.1 Pembacaan Citra dan Konversi BGR ke RGB

Citra dibaca menggunakan cv2.imread() yang secara default menghasilkan format BGR. Karena Matplotlib menampilkan citra dalam format RGB. Output sebelum konversi (BGR):

```
[1]: import cv2
import numpy as np
import matplotlib.pyplot as plt

[2]: citra = cv2.imread("manipulasi_warna.jpeg")

[3]: plt.imshow(citra)
plt.axis('off')
plt.show()
```



3.1.2 Manipulasi Kontras Citra Warna (Alpha = 1.2)

Manipulasi kontras pada citra warna dilakukan secara manual menggunakan iterasi piksel dengan nilai alpha (α) = 1.2, yang berarti setiap nilai piksel dikalikan 1.2. Hasil perkalian dibatasi pada rentang [0, 255] menggunakan casting ke np.uint8.

```
[11]: citra_rgb = cv2.cvtColor(citra, cv2.COLOR_BGR2RGB)

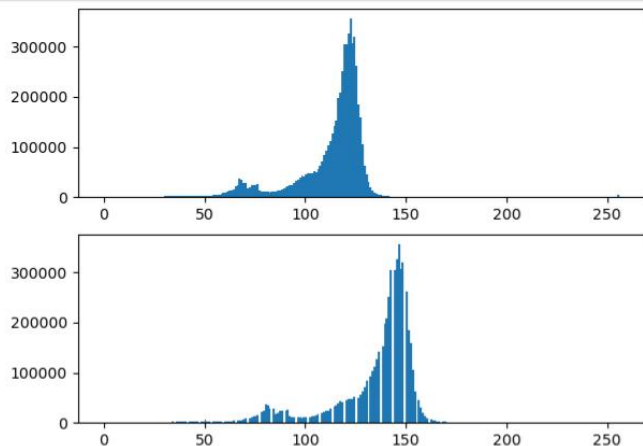
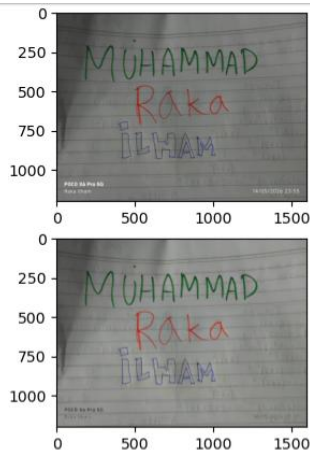
[12]: alpha = 1.2
citra_kontras = np.zeros((baris, kolom, 3))

for x in range(baris):
    for y in range(kolom):
        gx = citra_rgb[x,y] * alpha
        citra_kontras[x,y] = gx

citra_kontras = citra_kontras.astype(np.uint8)

fig, axs = plt.subplots(2,2, figsize=(15,5))
axs[0,0].imshow(rgb)
axs[0,1].hist(rgb.ravel(),256, [0,256])
axs[1,0].imshow(citra_kontras)
axs[1,1].hist(citra_kontras.ravel(),256,[0,256])

plt.show()
```



Dari perbandingan citra dan histogram di atas, dapat dianalisis:

- Citra asli: Histogram terpusat di sekitar intensitas 110-140, dengan puncak tertinggi di sekitar nilai 125. Distribusi yang mengumpul di area tengah-kanan menunjukkan citra cenderung terang karena didominasi warna latar kertas putih.
- Citra hasil kontras ($\alpha=1.2$): Secara visual, citra tampak lebih cerah dan tulisan berwarna terlihat lebih jenuh. Pada histogram, seluruh distribusi bergeser ke kanan mendekati nilai 140-160, dan sejumlah piksel yang sebelumnya berada di bawah 255 kini mencapai nilai maksimum 255, ditandai dengan lonjakan kecil di ujung kanan histogram. Hal ini sesuai dengan prinsip perkalian nilai piksel oleh faktor $\alpha > 1$ yang meningkatkan intensitas seluruh piksel secara proporsional.

3.1.3 Segmentasi Warna Menggunakan Masking HSV

Segmentasi dilakukan dengan mengonversi citra ke ruang warna HSV, kemudian mendefinisikan rentang nilai (*lower* dan *upper bound*) untuk masing-masing warna target: biru, merah, dan hijau. Fungsi `cv2.inRange()` menghasilkan mask biner, dan piksel di luar rentang warna yang ditentukan diisi dengan nilai putih [255, 255, 255].


```
[13]: hsv = cv2.cvtColor(citra, cv2.COLOR_BGR2HSV)

[14]: lower_blue = np.array([100, 50, 50])
upper_blue = np.array([140, 255, 255])
mask_blue = cv2.inRange(hsv, lower_blue, upper_blue)

lower_green = np.array([35, 50, 50])
upper_green = np.array([85, 255, 255])
mask_green = cv2.inRange(hsv, lower_green, upper_green)

lower_red1 = np.array([0, 50, 50])
upper_red1 = np.array([10, 255, 255])
lower_red2 = np.array([160, 50, 50])
upper_red2 = np.array([180, 255, 255])
mask_red1 = cv2.inRange(hsv, lower_red1, upper_red1)
mask_red2 = cv2.inRange(hsv, lower_red2, upper_red2)
mask_red = cv2.bitwise_or(mask_red1, mask_red2)

hasil_biru = citra_rgb.copy()
hasil_biru[mask_blue == 0] = [255, 255, 255]

hasil_hijau = citra_rgb.copy()
hasil_hijau[mask_green == 0] = [255, 255, 255]

hasil_merah = citra_rgb.copy()
hasil_merah[mask_red == 0] = [255, 255, 255]

plt.figure(figsize=(15, 10))

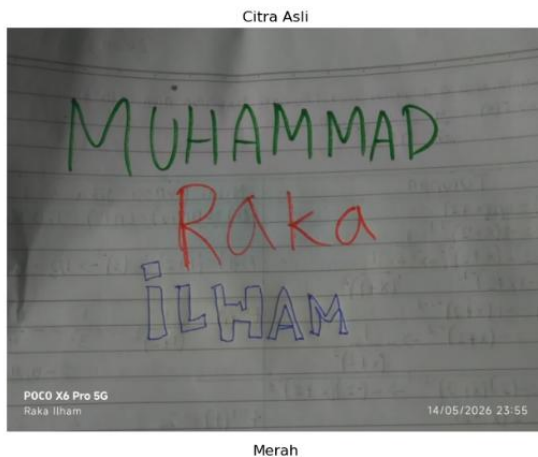
plt.subplot(2, 2, 1)
plt.imshow(citra_rgb)
plt.title('Citra Asli')
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(hasil_biru)
plt.title('Biru')
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(hasil_merah)
plt.title('Merah')
plt.axis('off')
|
plt.subplot(2, 2, 4)
plt.imshow(hasil_hijau)
```

```
plt.subplot(2, 2, 4)
plt.imshow(hasil_hijau)
plt.title('Hijau')
plt.axis('off')

plt.tight_layout()
plt.show()
```



Citra Asli

Biru

Merah

Hijau

Hasil segmentasi per warna:

- Segmentasi Biru: Berhasil mengisolasi tulisan "ILHAM" yang ditulis dengan tinta biru/ungu. Latar belakang kertas berwarna putih bersih, menunjukkan mask bekerja efektif dalam memisahkan piksel biru dari latar.
- Segmentasi Merah: Berhasil mengisolasi tulisan "Raka" yang ditulis dengan tinta merah. Perlu dua rentang HSV (0-10 dan 160-180) karena warna merah dalam ruang HSV berada di dua ujung spektrum *hue*, kemudian digabungkan dengan `cv2.bitwise_or()`.
- Segmentasi Hijau: Berhasil mengisolasi tulisan "MUHAMMAD" yang ditulis dengan tinta hijau. Rentang *hue* hijau (35-85) cukup lebar sehingga mampu menangkap variasi warna hijau pada tulisan.

3.1.4 Kombinasi Masking (Citra Biner)

Pada tahap ini, mask-mask individual dikombinasikan secara bertahap menggunakan operasi `cv2.bitwise_or()` untuk melihat akumulasi deteksi warna dalam bentuk citra biner.

```

•[15]: lower_blue = np.array([100, 50, 50])
upper_blue = np.array([140, 255, 255])
mask_blue = cv2.inRange(hsv, lower_blue, upper_blue)

lower_green = np.array([35, 50, 50])
upper_green = np.array([85, 255, 255])
mask_green = cv2.inRange(hsv, lower_green, upper_green)

lower_red1 = np.array([0, 50, 50])
upper_red1 = np.array([10, 255, 255])
lower_red2 = np.array([160, 50, 50])
upper_red2 = np.array([180, 255, 255])
mask_red1 = cv2.inRange(hsv, lower_red1, upper_red1)
mask_red2 = cv2.inRange(hsv, lower_red2, upper_red2)
mask_red = cv2.bitwise_or(mask_red1, mask_red2)

mask_none = np.zeros_like(mask_blue)
mask_b = mask_blue

mask_rb = cv2.bitwise_or(mask_red, mask_blue)
mask_rgb = cv2.bitwise_or(mask_rb, mask_green)

plt.figure(figsize=(15, 10))
plt.subplot(2, 2, 1)
plt.imshow(mask_none, cmap='gray')
plt.title('NONE')

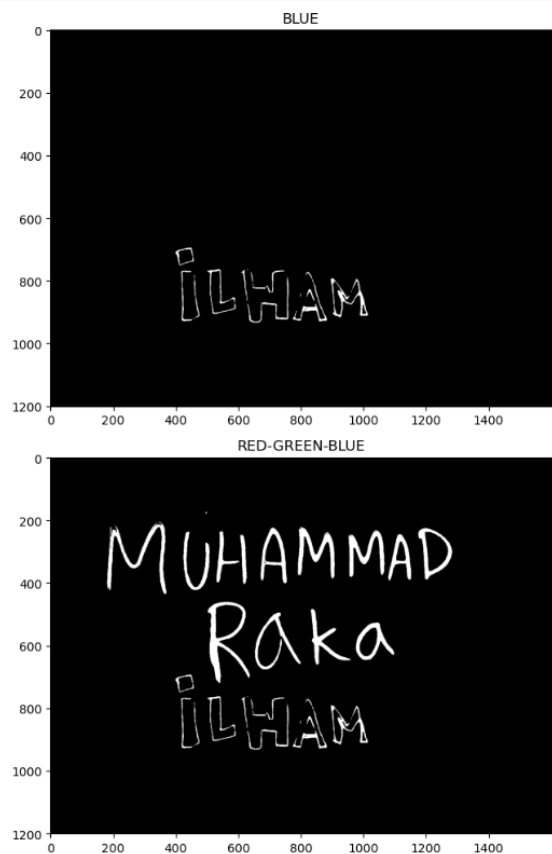
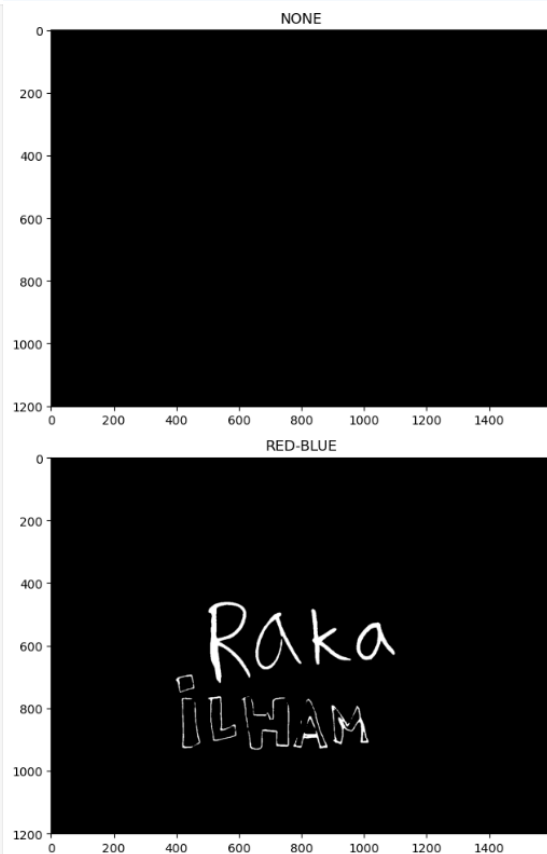
plt.subplot(2, 2, 2)
plt.imshow(mask_b, cmap='gray')
plt.title('BLUE')

plt.subplot(2, 2, 3)
plt.imshow(mask_rb, cmap='gray')
plt.title('RED-BLUE')

plt.subplot(2, 2, 4)
plt.imshow(mask_rgb, cmap='gray')
plt.title('RED-GREEN-BLUE')

plt.tight_layout()
plt.show()

```



Analisis tiap kondisi mask:

- NONE: Seluruh piksel bernilai 0 (hitam) karena mask diisi dengan np.zeros_like(). Tidak ada warna yang terdeteksi.

- BLUE: Hanya tulisan "ILHAM" yang tampil putih di atas latar hitam, sesuai dengan deteksi warna biru saja.
- RED-BLUE: Tulisan "Raka" (merah) dan "ILHAM" (biru) keduanya tampil putih. Kombinasi dua mask dilakukan dengan bitwise_or sehingga piksel yang termasuk merah atau biru keduanya aktif bernilai 255.
- RED-GREEN-BLUE: Ketiga tulisan "MUHAMMAD", "Raka", dan "ILHAM" seluruhnya tampil putih, menunjukkan bahwa semua piksel berwarna pada citra berhasil terdeteksi dan disegmentasi secara menyeluru

3.2 Soal 2 Operasi Piksel pada Citra Grayscale

Citra yang digunakan: RAKA.jpeg foto portrait seseorang dengan latar belakang langit sore dan atap rumah, berukuran 1280×720 piksel.

3.2.1 Pembacaan Citra dan Konversi ke Grayscale

Citra RAKA.jpeg dibaca, dikonversi ke RGB, kemudian dikonversi ke grayscale menggunakan cv2.cvtColor(rgb, cv2.COLOR_BGR2GRAY).

operasi pixel

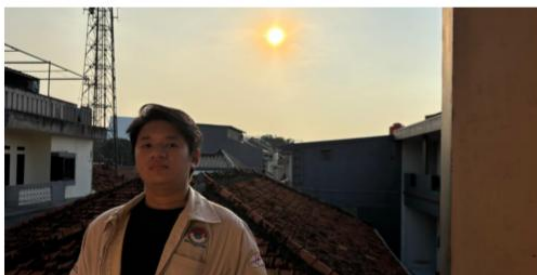
```
[16]: img = cv2.imread("RAKA.jpeg")
```

```
[17]: plt.imshow(img)
plt.axis('off')
plt.show()
```



```
[18]: rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
[40]: plt.imshow(rgb)
plt.axis('off')
plt.show()
```



Citra asli menampilkan foto portrait dengan latar langit sore berwarna kuning-oranye keemasan dan sumber cahaya matahari yang terlihat jelas di area atas. Kondisi pencahayaan foto ini memiliki kontras tinggi antara area matahari yang sangat terang dengan area bayangan gelap di wajah dan baju subjek.

3.2.2 Citra Grayscale dan Visualisasi Histogram

```
[19]: print(img.shape)
(720, 1280, 3)

[42]: gray = cv2.cvtColor(rgb, cv2.COLOR_BGR2GRAY)
cv2.imshow("Gray", gray)
cv2.waitKey(5000)
cv2.destroyAllWindows()

[22]: print(gray.shape)
(720, 1280)

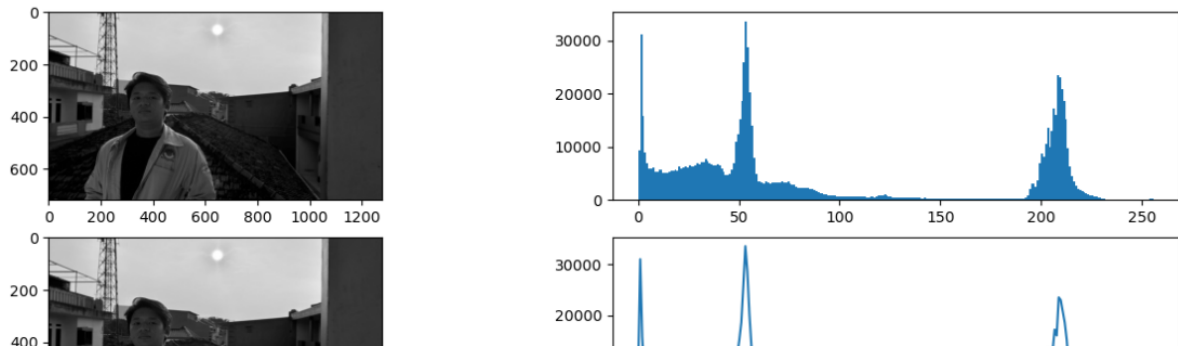
[47]: fig, axes = plt.subplots(2, 2, figsize = (15, 5))

# Tambahkan cmap='gray' di sini
axes[0,0].imshow(gray, cmap="gray")
axes[0,1].hist(gray.ravel(), 256, [0, 256])

hist = cv2.calcHist([gray], [0], None, [256], [0, 256])
# Dan tambahkan juga di sini
axes[1,0].imshow(gray, cmap="gray")
axes[1,1].plot(hist)

plt.show()

C:\Users\kurok\AppData\Local\Temp\ipykernel_28796\914996203.py:5: MatplotlibDeprecationWarning: Passing the range parameter of hist() positionally is deprecated since Matplotlib 3.9; the parameter will become keyword-only in 3.11.
axes[0,1].hist(gray.ravel(), 256, [0, 256])
```



Setelah konversi ke grayscale, citra tampil dalam nuansa abu-abu. Histogram ditampilkan dengan dua cara: menggunakan `gray.ravel()` dengan `plt.hist()` yang menghasilkan tampilan batang (*bar chart*), dan menggunakan `cv2.calcHist()` dengan `plt.plot()` yang menghasilkan tampilan kurva (*line chart*). Keduanya menghasilkan bentuk distribusi yang identik, hanya berbeda gaya visualisasi.

Dari histogram terlihat dua puncak yang signifikan:

- Puncak pertama di sekitar intensitas 0-10 dan 40-60, merepresentasikan piksel gelap dari area rambut, baju hitam, dan bayangan.
- Puncak kedua di sekitar intensitas 200-215, merepresentasikan piksel terang dari area langit dan sumber matahari.
- Area tengah (intensitas 100-180) relatif kosong, mengindikasikan foto ini memiliki distribusi piksel bimodal sebagian besar piksel terbagi menjadi sangat gelap atau sangat terang, dengan sedikit piksel di intensitas menengah.

3.2.3 Penyesuaian Kecerahan (Beta = 20) Manual Loop

Penyesuaian kecerahan dilakukan secara manual dengan iterasi *nested loop* per piksel. Setiap piksel ditambahkan nilai beta (β) = 20 menggunakan rumus $g(x,y) = f(x,y) + \beta$, dengan pembatasan nilai menggunakan fungsi `min(255, gxy)` agar tidak melewati nilai maksimum.

```
[49]: beta = 20
baris, kolom = gray.shape

# Buat img_cerah dengan 2 dimensi saja (karena gray itu 2D)
img_cerah = np.zeros((baris, kolom), dtype=np.uint8)

for x in range(baris):
    for y in range(kolom):
        # Gunakan int() agar tidak overflow saat proses tambah
        gx = int(gray[x, y]) + beta
        # Batasi agar tidak Lewat dari 255
        img_cerah[x, y] = min(255, gx)

img_cerah = img_cerah.astype(np.uint8)

fig, axs = plt.subplots(2, 2, figsize = (15, 5))

axs[0,0].imshow(gray, cmap='gray')
axs[0,1].hist(gray.ravel(), 256, [0, 256])
axs[1,0].imshow(img_cerah, cmap='gray')
axs[1,1].hist(img_cerah.ravel(), 256, [0, 256])
```

C:\Users\kurok\AppData\Local\Temp\ipykernel_28796\2281829742.py:19: MatplotlibDeprecationWarning: Passing the range parameter of hist() positionally is deprecated since Matplotlib 3.9; the parameter will become keyword-only in 3.11.

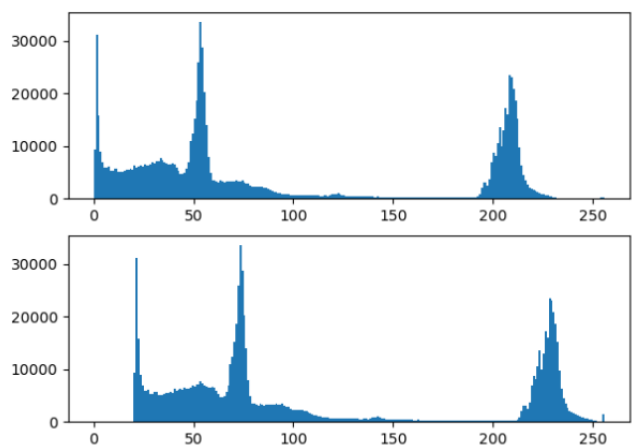
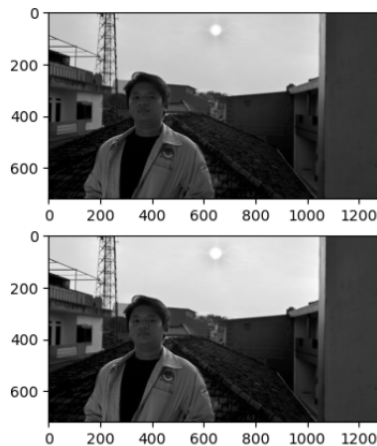
axs[0,1].hist(gray.ravel(), 256, [0, 256])

C:\Users\kurok\AppData\Local\Temp\ipykernel_28796\2281829742.py:21: MatplotlibDeprecationWarning: Passing the range parameter of hist() positionally is deprecated since Matplotlib 3.9; the parameter will become keyword-only in 3.11.

axs[1,1].hist(img_cerah.ravel(), 256, [0, 256])

```
[49]: (array([[ 0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,
  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,
  0.,  0.,  0.,  0., 9313., 31054., 15715., 8925.,
6987., 5975., 5909., 6143., 5282., 5383., 5725., 5620.,
5162., 5162., 5156., 5213., 5488., 5561., 5636., 5505.,
6263., 5908., 6139., 6360., 6158., 6425., 6215., 6396.,
6502., 6900., 6634., 7090., 7128., 7773., 7391., 6860.,
6752., 6518., 6507., 6694., 6490., 5894., 5251., 4732.,
4765., 4914., 5701., 6968., 10853., 12332., 15194., 18656.,
25877., 33623., 28676., 20300., 13897., 7897., 4813., 3534.,
3391., 3198., 3023., 3504., 3246., 3070., 3318., 3336.,
3337., 3338., 3271., 3386., 3334., 3260., 3437., 3165.,
2943., 2783., 2547., 2337., 2265., 2277., 2203., 2262.,
2291., 2139., 2122., 1802., 1691., 1471., 1302., 1254.,
1041., 970., 949., 872., 814., 773., 751., 716.,
738., 614., 641., 647., 630., 617., 613., 641.,
612., 596., 613., 658., 639., 541., 554., 615.,
588., 540., 607., 830., 814., 928., 1023., 830.,
11., 12., 13., 14., 15., 16., 17., 18., 19., 20., 21.,
22., 23., 24., 25., 26., 27., 28., 29., 30., 31., 32.,
33., 34., 35., 36., 37., 38., 39., 40., 41., 42., 43.,
44., 45., 46., 47., 48., 49., 50., 51., 52., 53., 54.,
55., 56., 57., 58., 59., 60., 61., 62., 63., 64., 65.,
66., 67., 68., 69., 70., 71., 72., 73., 74., 75., 76.,
77., 78., 79., 80., 81., 82., 83., 84., 85., 86., 87.,
88., 89., 90., 91., 92., 93., 94., 95., 96., 97., 98.,
99., 100., 101., 102., 103., 104., 105., 106., 107., 108., 109.,
110., 111., 112., 113., 114., 115., 116., 117., 118., 119., 120.,
121., 122., 123., 124., 125., 126., 127., 128., 129., 130., 131.,
132., 133., 134., 135., 136., 137., 138., 139., 140., 141., 142.,
143., 144., 145., 146., 147., 148., 149., 150., 151., 152., 153.,
154., 155., 156., 157., 158., 159., 160., 161., 162., 163., 164.,
165., 166., 167., 168., 169., 170., 171., 172., 173., 174., 175.,
176., 177., 178., 179., 180., 181., 182., 183., 184., 185., 186.,
187., 188., 189., 190., 191., 192., 193., 194., 195., 196., 197.,
198., 199., 200., 201., 202., 203., 204., 205., 206., 207., 208.,
209., 210., 211., 212., 213., 214., 215., 216., 217., 218., 219.,
220., 221., 222., 223., 224., 225., 226., 227., 228., 229., 230.,
231., 232., 233., 234., 235., 236., 237., 238., 239., 240., 241.,
242., 243., 244., 245., 246., 247., 248., 249., 250., 251., 252.,
253., 254., 255., 256.])).

<BarContainer object of 256 artists>)
```



Hasil analisis perbandingan:

- Secara visual: Citra tampak lebih terang secara merata di seluruh area. Area wajah yang sebelumnya cenderung gelap akibat pencahayaan *backlit* terlihat sedikit lebih jelas.
- Pada histogram: Seluruh distribusi bergeser sejauh +20 ke kanan (dari rentang 0–255 menjadi 20–255 untuk piksel non-saturasi). Puncak pertama yang semula di sekitar intensitas 0–60 kini bergeser ke 20–80, dan puncak kedua dari sekitar 200–215 bergeser mendekati 220–235. Pergeseran ini konsisten dengan penambahan konstanta $\beta = 20$ pada seluruh piksel.

3.2.4 Penyesuaian Kontras (Alpha = 1.2) Manual Loop

Penyesuaian kontras dilakukan secara manual dengan mengalikan setiap piksel grayscale dengan nilai alpha (α) = 1.2, kemudian dibatasi dengan `np.clip()` pada rentang [0, 255].

```
[51]: alpha = 1.2
      baris, kolom = gray.shape[:2] # Mengambil dimensi dari gambar input

      # Inisialisasi dengan nol
      gray_kontras = np.zeros((baris, kolom), dtype=np.float32)

      for x in range(baris):
          for y in range(kolom):

              gx = gray[x, y] * alpha
              gray_kontras[x, y] = np.clip(gx, 0, 255)

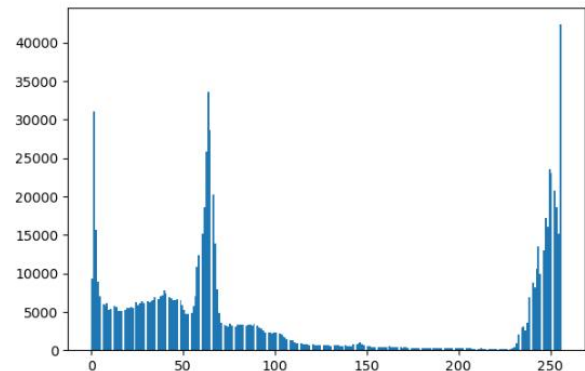
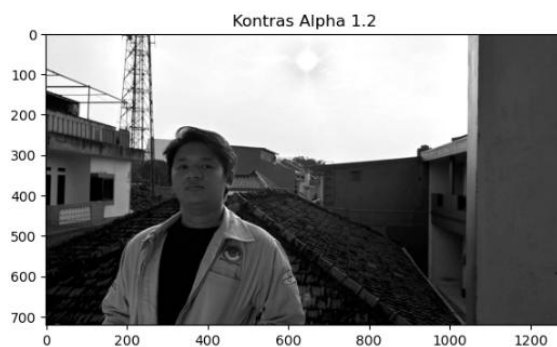
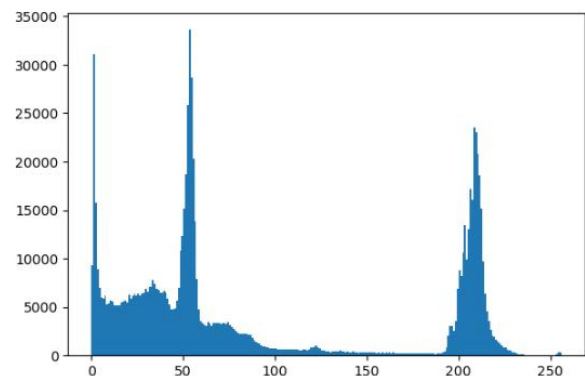
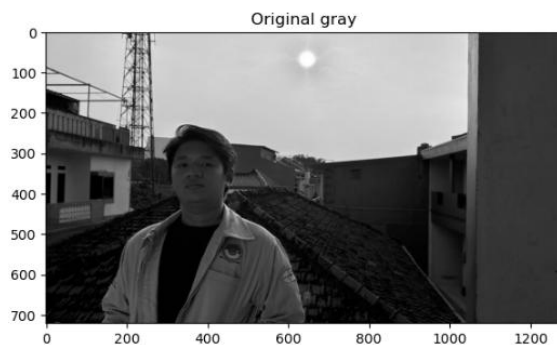
      gray_kontras = gray_kontras.astype(np.uint8)

      # Visualisasi
      fig, axes = plt.subplots(2, 2, figsize=(15, 10))
      axes[0,0].imshow(gray, cmap='gray')
      axes[0,0].set_title("Original gray")
      axes[0,1].hist(gray.ravel(), 256, [0, 256])

      axes[1,0].imshow(gray_kontras, cmap='gray')
      axes[1,0].set_title(f"Kontras Alpha {alpha}")
      axes[1,1].hist(gray_kontras.ravel(), 256, [0, 256])

      plt.show()
```

C:\Users\kurok\AppData\Local\Temp\ipykernel_28796\3973011278.py:20: MatplotlibDeprecationWarning: Passing the range parameter of hist() positionally is deprecated since Matplotlib 3.9; the parameter will become keyword-only in 3.11.
 axes[0,1].hist(gray.ravel(), 256, [0, 256])
 C:\Users\kurok\AppData\Local\Temp\ipykernel_28796\3973011278.py:24: MatplotlibDeprecationWarning: Passing the range parameter of hist() positionally is deprecated since Matplotlib 3.9; the parameter will become keyword-only in 3.11.
 axes[1,1].hist(gray_kontras.ravel(), 256, [0, 256])



Hasil analisis perbandingan:

Secara visual: Citra tampak lebih kontras area terang menjadi semakin terang, dan area gelap tetap gelap. Efek ini memperjelas perbedaan antara objek terang (langit, matahari) dengan objek gelap (baju, rambut).

Pada histogram: Distribusi piksel yang semula terpusat di dua puncak kini menyebar lebih luas. Puncak area gelap (0-60) menyebar lebih ke kanan karena nilainya dikali 1.2 (misalnya piksel bernilai 50 menjadi 60), sementara piksel yang sebelumnya mendekati 255 kini mencapai dan menumpuk di nilai 255 karena ter-clip (ditandai lonjakan besar di ujung kanan histogram). Hal ini khas pada peningkatan kontras dengan faktor $\alpha > 1$.

3.2.5 Perbandingan Empat Kondisi (Original, Cerah, Kontras, Combo) Vectorized

Pada bagian akhir, keempat kondisi dibandingkan sekaligus menggunakan operasi vektorisasi NumPy (`np.clip()`) yang jauh lebih efisien dibandingkan manual loop, namun menghasilkan output yang identik secara matematis.

Pada kondisi Combo, histogram menunjukkan sebagian besar piksel terkonsentrasi di nilai 255 (saturasi penuh) akibat efek gabungan perkalian dan penambahan. Secara visual, area langit dan matahari pada citra menjadi hampir sepenuhnya putih (*blown out*), sementara area gelap masih terjaga karena nilai rendah dikalikan 1.2 dan ditambah 20 masih tetap di bawah 255.

```
[46]: alpha = 1.2 # Nilai kontras
      beta = 20 # Nilai kecerahan

      gray_float = gray.astype(np.float32)
      gray_cerah = np.clip(gray_float + beta, 0, 255).astype(np.uint8)
      gray_kontras = np.clip(gray_float * alpha, 0, 255).astype(np.uint8)
      gray_combo = np.clip((gray_float * alpha) + beta, 0, 255).astype(np.uint8)
      fig, axs = plt.subplots(2, 4, figsize=(20, 8))

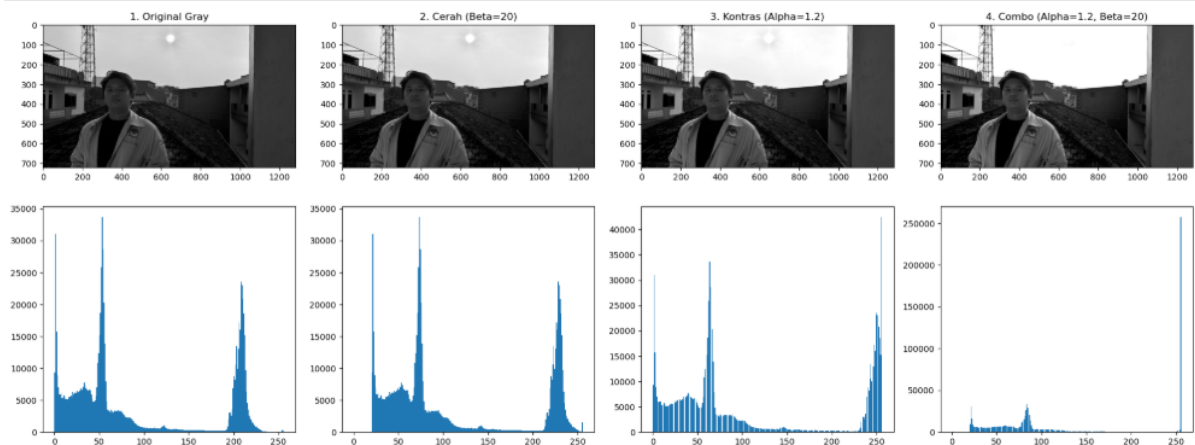
      axs[0,0].imshow(gray, cmap='gray')
      axs[0,0].set_title("1. Original Gray")
      axs[1,0].hist(gray.ravel(), bins=256, range=[0, 256])

      axs[0,1].imshow(gray_cerah, cmap='gray')
      axs[0,1].set_title(f"2. CERAH (Beta={beta})")
      axs[1,1].hist(gray_cerah.ravel(), bins=256, range=[0, 256])

      axs[0,2].imshow(gray_kontras, cmap='gray')
      axs[0,2].set_title(f"3. Kontras (Alpha={alpha})")
      axs[1,2].hist(gray_kontras.ravel(), bins=256, range=[0, 256])

      axs[0,3].imshow(gray_combo, cmap='gray')
      axs[0,3].set_title(f"4. Combo (Alpha={alpha}, Beta={beta})")
      axs[1,3].hist(gray_combo.ravel(), bins=256, range=[0, 256])

      plt.tight_layout()
      plt.show()
```



BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan landasan teori dan hasil praktikum UTS Pengolahan Citra Digital yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut.

1. Citra digital direpresentasikan sebagai array dua dimensi piksel dengan nilai intensitas 0–255. OpenCV membaca citra dalam format BGR sehingga diperlukan konversi ke RGB menggunakan `cv2.cvtColor()` sebelum ditampilkan dengan Matplotlib agar warna tampil sesuai aslinya.
2. Segmentasi warna berbasis masking HSV terbukti efektif dalam mengisolasi warna tertentu dari citra. Dengan mendefinisikan rentang lower dan upper bound pada ruang warna HSV, ketiga warna target merah, hijau, dan biru berhasil diekstrak secara terpisah maupun dikombinasikan bertahap menggunakan operasi `cv2.bitwise_or()`.
3. Manipulasi kecerahan dengan penambahan nilai beta ($\beta = 20$) menggeser seluruh distribusi histogram ke kanan secara seragam, sementara manipulasi kontras dengan perkalian alpha ($\alpha = 1.2$) melebarkan distribusi histogram dan menyebabkan penumpukan piksel di nilai 255 akibat efek clipping. Kombinasi keduanya menghasilkan citra yang paling terang sekaligus paling kontras.
4. Pendekatan manual (nested loop per piksel) dan vektorisasi NumPy menghasilkan output yang identik secara matematis, namun vektorisasi jauh lebih efisien secara komputasi untuk citra berukuran besar.
5. Histogram citra merupakan alat analisis yang sangat berguna untuk membaca karakteristik citra distribusi yang miring ke kiri menandakan citra gelap, ke kanan menandakan citra terang, dan distribusi bimodal seperti pada citra RAKA.jpeg menunjukkan adanya kontras tinggi antara objek gelap dan terang dalam satu frame.

DAFTAR PUSTAKA

- Fatimatuzzahro, S., & Yuliantari, R. V. (2022). Peningkatan Kualitas Citra pada Foto Sejarah Menggunakan Metode Histogram Equalization dan Intensity Adjustment. *Journal of Applied Electrical Engineering*, 5(2), 36–42.
- Syahidi, A. A. (2024). *Pengolahan Citra Digital dan Dasar Visi Komputer dengan Python*. Poliban Press.
- Dr. Darma Rusjdi ST MKom & Rizqia Cahyaningtyas, ST., M.Kom (2026) *Pengolahan citra digital modul bab 1-5*